# COMP3011 Technical Report

## Football Analytics & Community REST API

**Module:** COMP3011 Web Services and Web Data
**Name:** Hany Song
**Student ID:** 202049082

**Public GitHub Repository Link**
https://github.com/20210330-12/COMP3011-Coursework1

**API Documentation PDF Link**
https://github.com/20210330-12/COMP3011-Coursework1/blob/main/COMP3011_API_Documentation.pdf

**Presentation Slides**
https://shorturl.at/4TRVx

**Abstract** This project presents the design and implementation of a RESTful web API that provides football analytics and community discussion functionality. The system integrates publicly available football datasets and exposes structured endpoints that allow users to retrieve information about teams, players, matches, and player performance statistics. In addition to analytical features, the API also includes community interaction capabilities where authenticated users can create posts, write comments, and interact with content through a like system. The backend was implemented using Python, Django, and Django REST Framework, with SQLite used as the relational database. The system was deployed on an AWS EC2 instance using Nginx and Gunicorn, providing external access to the API. The project also incorporates automated API documentation generated from the OpenAPI schema and a set of automated tests to verify endpoint functionality. The implementation demonstrates key software engineering principles including RESTful API design, database modelling, authentication mechanisms, and data-driven analytics. The resulting system provides a practical example of how real-world datasets can be integrated into a scalable web service architecture while supporting both analytical queries and user-generated content.

## 1. Project Overview

This project implements a data-driven REST API for football analytics and community discussion. The system combines two related domains. The first is a football statistics service that provides access to teams, players, matches, player appearances, and analytical summaries. The second is a community platform that allows authenticated users to register, log in, create posts, write comments, and interact through likes.

The project was designed to align with the coursework brief's emphasis on database-backed API development, software engineering principles, external deployment, and reflective technical justification. The chosen problem domain also fits the brief's example of a sports performance and match statistics API supported by public sports datasets and analytical endpoints.

The API is externally deployed on AWS EC2 and documented through an exported OpenAPI-based PDF referenced from the README, which satisfies the documentation requirement in the brief.

## 2. Design and Architectural Choices

The system was implemented as a RESTful API using Django and Django REST Framework. This stack was selected because it provides a strong balance between rapid development and structured engineering practice. Django offers a mature ORM, migrations, and a clean project structure, while Django REST Framework provides serializers, generic views, authentication, throttling, pagination, and automatic schema generation. These features made it possible to focus on designing resources and behaviour instead of writing repetitive infrastructure code.

The architecture is modular and separated into three main applications:

- accounts for authentication and user profiles
- football for sports data and analytics
- community for posts, comments, and likes

This modular structure improves maintainability and makes the system easier to explain, test, and extend. It also reflects the clean code and modular design qualities associated with higher-grade work in the marking criteria.

The deployed architecture is a layered system:

**Client → Nginx → Gunicorn → Django REST Framework → SQLite**

Nginx acts as a reverse proxy, Gunicorn serves the Django application, and Django handles request routing, business logic, and database access. This is a standard and professional deployment pattern for Python web services and demonstrates that the project is not limited to local execution.

## 3. Technology Stack Justification

Python was chosen because it is highly suitable for API development and data processing, and because it integrates well with both Django and CSV-based import workflows. Django REST Framework was chosen instead of lower-level alternatives because the project needed authentication, validation, pagination, permissions, and documentation support, all of which DRF provides in a coherent way.

SQLite was chosen as the database engine. For a coursework-scale system, SQLite is practical because it is lightweight, requires minimal configuration, and works smoothly with Django. Although a larger production deployment would normally use PostgreSQL, SQLite was appropriate here because it reduced operational complexity while still satisfying the brief's requirement for an SQL-backed system. The brief explicitly allows SQL databases and expects students to justify their stack choices rather than use a specific database product.

For deployment, AWS EC2 was chosen to provide a real external hosting environment rather than a local-only demonstration. This allowed the project to satisfy the external deployment expectation in the marking guidance and also provided useful practical experience in production configuration.

## 4. Data Model and Dataset Integration

The project uses a public Kaggle dataset, player-scores, and imports the following files into the database:

- clubs.csv
- players.csv
- games.csv
- appearances.csv

The brief encourages the use of public datasets and specifically lists Kaggle as an

appropriate source. It also recommends using AI tools to help discover datasets and generate import scripts where appropriate.

The main football models are Team, Player, Match, and Appearance. The most important design choice is the Appearance model, which links a player, a team, and a match while storing performance-level data such as minutes played, goals, assists, cards, and starting status. This model is the analytical bridge of the system because it makes aggregation straightforward. Instead of storing only match-level outcomes, the database also stores player-level contributions within matches.

A custom Django management command was implemented to import dataset files into the correct dependency order. Teams are imported before players and matches, and appearances are imported last so that foreign key relationships already exist. This import approach keeps the process reproducible and makes it easy to rebuild the database from source files.

The community side of the schema adds UserProfile, Post, and Comment, connecting football data to user-generated discussion. This was an intentional design decision to move the project beyond a read-only sports statistics API and introduce authenticated CRUD behaviour and permissions.

## 5. API Implementation and Analytical Features

The API exposes authentication endpoints, football data endpoints, analytics endpoints, and community endpoints. Resource-based routing was used throughout, with examples such as /api/teams/, /api/players/, /api/matches/, /api/posts/, and /api/comments/. Nested resources are also provided where useful, such as team players, team matches, player matches, and player appearances.

Analytical endpoints include top scorers, top assists, most minutes played, team top scorers, and league standings. These are computed dynamically from stored database records rather than being hard-coded tables. This demonstrates database querying and aggregation more effectively than returning static data.

The standings endpoint calculates league table values from match results. For each match in a selected season, both teams have their played count updated, goals for and against are accumulated, and points are awarded according to the standard scoring system of three points for a win and one point for a draw. Teams are then sorted by points, goal difference, goals scored, and finally by name for stable ordering. This design shows how raw match records can be transformed into higher-level analytics.

The API also implements token-based authentication, pagination, query-parameter filtering, and rate limiting. These features improve usability and robustness and help the API behave more like a real-world service rather than a minimal coursework prototype.

## 6. Testing Approach, Challenges, and Lessons Learned

Testing was carried out using Django and Django REST Framework's testing tools. The automated test suite covers authentication flows, football resource retrieval, filtering behaviour, analytics endpoints, post and comment CRUD operations, and author-based permissions. This verifies both the correctness of the API logic and the expected HTTP status behaviour.

One of the main practical challenges arose during external deployment. The EC2 instance initially experienced instability because the small t3.nano environment had very limited memory, and the Linux OOM

killer terminated Gunicorn under memory pressure. This was resolved by reducing Gunicorn worker usage, correcting ALLOWED_HOSTS, and improving service robustness. This was a valuable lesson in the difference between code that works locally and code that runs reliably in production.

A second challenge involved documentation quality. Auto-generated OpenAPI output was useful, but not always sufficient by itself because some example error responses and overview information were unclear or incomplete. This was addressed by adding a manually written front section to the documentation PDF covering base URL, authentication, filtering, pagination, and representative error responses. The lesson here was that generated documentation is strongest when combined with concise human-written context.

A broader lesson from the project is that API quality depends not only on endpoint count, but on coherent structure, predictable behaviour, documentation, and testing discipline.

## 7. Limitations and Future Development

The project has several limitations. First, the imported football dataset is not perfectly complete, which means some computed outputs such as standings may differ from official historical tables if underlying fixture data is missing. This is a limitation of the source data rather than the analytical logic.

Second, the system currently focuses on backend API functionality and does not include a dedicated frontend client. Demonstration is therefore performed through Swagger/OpenAPI documentation and direct API interaction.

Third, SQLite is appropriate for this coursework project but would not be the best choice for a larger multi-user production environment. A future version would likely migrate to PostgreSQL for stronger concurrency and operational flexibility.

Future development could include live sports data refresh, caching of expensive analytics queries, more advanced metrics such as per-90 performance indicators or expected goals, moderation tools for community content, and a frontend dashboard for visualization.

## 8. Generative AI Declaration and Reflective Analysis

This assessment is classified as a **Green Light assessment**, meaning that the use of Generative AI tools is permitted provided that their usage is clearly declared and evidenced. In accordance with the coursework brief, the AI tools used, their purposes, and representative conversation logs are documented.

The primary Generative AI tool used during this project was **ChatGPT (OpenAI)**, accessed through an interactive conversational interface. The tool was used throughout the project as a development assistant and exploratory reasoning partner, rather than as an automatic code generator. All AI-generated suggestions were reviewed, modified, and validated through manual implementation and testing before being incorporated into the final system.

ChatGPT was used methodologically across several stages of the development process.

**Project planning and problem exploration.**
At the early stage of the project, the AI tool was used to explore different API project

directions and evaluate potential datasets. Through iterative discussion, the project evolved from a simple sports statistics API concept into a combined football analytics and community interaction platform, integrating both data-driven endpoints and user-generated content features.

**Database modelling and API design.** During system design, ChatGPT was used to discuss schema structure and entity relationships for the football domain. These conversations helped refine the core models such as Team, Player, Match, and Appearance, as well as the community models UserProfile, Post, and Comment. The AI tool also assisted in reasoning about RESTful resource structures and endpoint organization.

**Implementation support and debugging.** Throughout development, the AI tool was used to troubleshoot implementation issues related to Django and Django REST Framework. This included assistance with serializer design, view logic, authentication behaviour, filtering strategies, and endpoint structuring. It was also used to diagnose deployment problems encountered during the AWS EC2 deployment phase, such as configuration issues involving Gunicorn workers and ALLOWED_HOSTS settings.

**Dataset integration and workflow design.**
ChatGPT was also used to support the planning of dataset import workflows. The project integrates multiple CSV files from a public football dataset, and the AI tool was used to reason about the correct dependency order for importing teams, players, matches, and appearances while maintaining foreign key integrity.

**Documentation and communication clarity.**
The AI tool was further used to refine the clarity and structure of documentation materials including the README file, API documentation introduction, technical report sections, and presentation slides. In this role, AI functioned primarily as a writing and structure assistant, helping improve technical communication.

Importantly, AI-generated outputs were not accepted automatically. Suggestions were critically evaluated, adjusted, and tested before integration. The final implementation decisions, architecture design, and debugging steps were carried out manually by the author.

The use of Generative AI in this project therefore focused on structured problem exploration, debugging assistance, and documentation refinement, rather than passive code generation. This reflects the coursework's emphasis on methodologically sound and reflective AI usage.

Examples of the ChatGPT conversations used during the development process are included as supplementary material in the submitted **ChatGPT_Conversation_Export.pdf**, as required by the coursework brief.

## 9. Conclusion

This project delivers a complete data-driven REST API that combines football analytics with community interaction. It demonstrates database design, RESTful resource modelling, authentication, filtering, pagination, automated testing, external deployment, and reflective technical decision-making. The final system satisfies the core technical goals of the coursework while also showing practical awareness of documentation quality, deployment constraints, and future extensibility.